

Lab2: Specifying a RESTful API

Suggested documentation

- [Lecture4 : REST/RESTful applications](#)
- [Swagger documentation](#)
- [Paypal REST API](#)

Objectives

- Get familiar with the REST principle;
- Learn how to use a REST api;
- Specify a REST api with OpenAPI;
- Implement your own REST server.

Getting started with an existing REST API

In the following, you will learn how to get started with an API describing the MOSIG Master 2. The API is available through the following link: <https://ppp.alwaysdata.net/>. The base route is `/`. If you attempt to do an HTTP GET request on <https://ppp.alwaysdata.net/>, the server can reply with a `200` success code in case the request was correctly constructed, or `404` (resource not found) or `500` (server error) indicating that an error originating from either the client or the server has been done.

Besides returning the corresponding resources, the API implements HATEOAS to guide the user in correctly using the API. The HATEOAS implementation that has been followed is [Paypal REST API](#).

The code below illustrates a response of the REST API when requesting <https://ppp.alwaysdata.net/> (GET HTTP request):

```
json
{
  "name": "Master 2 MOSIG",
  "links": [
    {
      "href": "/students/",
      "rel": "students",
      "method": "GET"
    },
    {
      "href": "/students/",
      "rel": "students",
      "method": "POST"
    },
    {
      "href": "/teachers/",
      "rel": "teachers",
      "method": "GET"
    },
    {
      "href": "/teachers/",
      "rel": "teachers",
      "method": "POST"
    },
    {
      "href": "/responsible/",
      "rel": "responsible",
      "method": "GET"
    },
    {
      "href": "/responsible/",
      "rel": "responsible",
      "method": "PUT"
    },
    {
      "href": "/modules/",
      "rel": "modules",
      "method": "GET"
    },
  ],
}
```

```
{
  "href": "/modules/",
  "rel": "modules",
  "method": "POST"
}
]
```

This response corresponds to the external representation of the resource `Master`, independently of its internal representation. It contains:

- Attributes (here, only the name is provided);
- A `links` section describing how to use the API from this endpoint.

Each link contains three attributes:

href

the URL endpoint to access a resource. The URL is relative to the base URL (<https://ppp.alwaysdata.net/>).

rel

The name of the relation used. It defines the relation that exists between the resources being fetched from an existing endpoint. The `self` value specifies that the request refers to the same resource as for the current URL endpoint.

method

The HTTP method into play.

1. Use Firefox to get familiar with the API;
2. Answer the following questions on the score sheet.

Firefox is enough to test the API, since it creates an HTTP GET request when you browse any web page. But, if you need to send other kinds of

HTTP requests (`PUT` , `POST` , `DELETE` , etc.), then, you would need to install an extension. For Firefox, many extensions are available, such as [RESTClient](#) or [RESTED](#). Such extensions are also available on other kind of web browsers. Those are useful to edit the header of an HTTP request. For example, to create a new student, you would need to add the following header:

`content-type: application/json` and provide the following content:

```
{  
  "studentNumber": "0123456789a",  
  "firstName": "dupont",  
  "lastName": "jean"  
}
```

json

Getting started with OpenAPI

The OpenAPI specification of the example above is written using OpenAPI. It describes all the possible routes that can be used by the user when traversing the API. The [specification is available here](#).

1. Open the specification in a text editor and identify the important sections;
2. Open the [Swagger editor](#) and copy the specification into it;
3. Observe the result in the right part of the editor and test some HTTP requests.

The code below shows the overall structure of the specification:

```
openapi: 3.0.2  
info:  
  version: 1.0.0  
  title: My API title
```

yaml

```
  description: My API description
  contact:
    name: "Nicolas Hili"
    email: nicolas.hili@univ-grenoble-alpes.fr
  servers:
  - url: https://ppp.alwaysdata.net/
  paths:
  ...
  components:
    schemas:
    ...
    parameters:
    ...
```

It is structured into different parts:

info

Contains some information about the API, including its version, a description, and contact information;

servers

Lists all servers to use the specification;

paths

Defines The set of endpoints. The attribute `tags` lets you organize the endpoints into different categories. The `in` attribute defines the type of parameters. We will limit ourself to two types of parameters in this lab session:

`path` and `query`. The `requestBody` attribute allows you to define the data needed by the request (for `POST` and `PUT` HTTP requests).

components

Defines a set of reusable components. This section is optional, but it allows for making the code shorter and more readable. It contains sub-sections for

`schemas`, `parameters`, `responses`, etc. More information are given in the [following page](#).

1. Have a look at the path `/teachers/` (`POST` method) and understand how the `requestBody` is used.
2. Have a look at the path `/modules/{code}/teachers/` (`GET` method) and understand how the parameters are being used.

Questions:

- How many parameters are defined by the path `/modules/{code}/teachers/` (`GET` method)?
- How many are shared with other routes?
- What is the value of the `in` attribute of the `offsetParam` parameter?
- What is the value of the `in` attribute of the `code` parameter?

Creating an API

Now that we have seen how an API is structured, we will structure our own. To do so, we will use the following template:

```
openapi: 3.0.2
info:
  version: 1.0.0
  title: Sociecity API
  description: Description of the API
  contact:
    name: First and last
    email: first.last@provider.com
servers:
- url: https://localhost:8080/
paths:
  /:
    get:
      tags: [society]
      summary: Some description
      responses:
```

yaml

```
200:
  description: Request correctly done
400:
  description: Error originating from the
```

Context: an API for a board game society

The house of games (*Maison des jeux*) in Grenoble wants to create an API for managing board game loans. The API must meet the following specification:

1. The association has a name, an address, a set of games, and a set of members;
2. Each game is identified by its name, description, date of purchase, condition (new, good condition, damaged) and availability (available, borrowed);
3. A game also has a category;
4. A category has a name and description, and a number indicating the number of games it contains;
5. It must be possible to list all of the society's games;
6. It must also be possible to list the categories of games and for each category, all the games it contains;
7. It must be possible to limit the number of games returned by a query;
8. It must be possible to find the name of the category from the name of the game;
9. In the event that the game is borrowed, it must be possible to find the member who borrowed it;
10. A member has a first name, last name, and telephone number.

Creating GET endpoints

From the description above, specify the endpoints, limited to the **GET** endpoints.

- It is required to specify the responses returned by the sever. Have a look at the [Swagger documentation](#).
- The **component** section is optional, but it can greatly make your code more organized;

- Do not forget parameters when appropriate;
- Create categories with the `tags` parameter to make your specification more readable.

[< Previous](#)

#1: Implementing a Web server in Node.JS

[Next >](#)

#3: Web application testing